Introduction to OOPS Programming

Section A: Concept Application

1. Compiler vs Interpreter in Food Delivery Application

For a food delivery application, a **compiler** is preferred during the final deployment stage because compiled programs execute faster and are more efficient for large applications. A compiler translates the entire source code into machine code before execution, which improves performance and reduces execution time.

It also detects many errors at once during compilation, making debugging easier before deployment. In contrast, an interpreter executes code line by line, which is slower and may stop immediately when an error occurs. Interpreters are useful during development and testing, but compilers are better for production applications where speed, security, and stability are important.

---

2. Programming Constructs for Employee Salary Calculation

To calculate employee salary with conditions such as bonus, tax, and overtime, a combination of the following programming constructs should be used:

- **Conditional statements (`if-else` or `switch`)** to check salary rules, tax slabs, and bonus eligibility.
- **Arithmetic operators** to calculate deductions and overtime pay.
- **Functions/Methods** to divide the program into smaller reusable modules like `calculateBonus()` or `calculateTax()`.
- **Loops** if salary data for multiple employees must be processed.

This combination improves efficiency, readability, and maintainability of the program.

---

3. Importance of Loops in Menu-Driven Applications

Loops are preferred in menu-driven console applications because they allow the program to repeat the menu automatically until the user chooses to exit. Without loops, the same code would need to be written repeatedly using conditional statements, making the program lengthy and inefficient.

The **`do-while` loop** is most suitable because:

- It executes the menu at least once before checking the condition.
- It is ideal when the user must see the menu before making a choice.
- It simplifies repeated execution until the exit option is selected.

---

4. Incorrect Average Calculation Using `int`

The probable cause is the use of the **`int` data type**, which stores only whole numbers. During average calculation, decimal values are truncated, leading to incorrect output.

Example:
If marks are `85`, `86`, and `87`:

$$\frac{85 + 86 + 87}{3} = 86.0$$

But with integer division, decimals may be lost.

Using `float` or `double` solves the issue because these data types can store decimal values accurately, producing the correct average result.

---

5. Program Crashes at Runtime

This is a **runtime error** because the program compiles successfully but fails during execution.

Two debugging practices to resolve it are:

1. **Using debugging tools or breakpoints**
   Breakpoints help identify the exact line where the crash occurs and allow checking variable values step by step.
2. **Testing with different inputs and exception handling**
   Trying various test cases helps identify invalid conditions, while exception handling prevents unexpected crashes and provides meaningful error messages.

---

6. Using Structure for Student Records

A **structure** is preferred over multiple arrays because it stores related data together in a single unit. In this case, name, marks, and grade of a student belong to the same record.

Advantages of using a structure:

- Improves data organization and readability.
- Reduces complexity compared to maintaining separate arrays.
- Makes data handling easier for adding, updating, or displaying records.
- Keeps all information of one student together, reducing chances of mismatch.

Example:
A structure can store:

- Student Name
- Marks
- Grade

as one complete student record instead of using different arrays for each field.

Section B: Practical Coding Tasks

1. Conditional Logic Implementation Design a "Study Mood Assistant" that:

● Takes user input: hours studied today

● Displays motivation messages based on conditions

● Uses if-else ladder or switch case

➔ here is the code:

```
let hours = parseInt(prompt("Enter hours studied today:"));

if (hours >= 8) {

    console.log("Excellent! You are highly productive today.");

}

else if (hours >= 5) {

    console.log("Good job! Keep maintaining consistency.");

}

else if (hours >= 2) {

    console.log("You can do better. Stay focused!");

}

else {
```

```
    console.log("Time to study seriously and avoid distractions.");
}
```

---

2. Loops + Arrays Create a program that:

● Accepts 7 days of screen time data

● Calculates and prints: ➔ Total screen time ➔ Average screen time

● Displays a warning if average exceeds healthy limit

➔ here is the code:

```
let screenTime = [];
let total = 0;
for (let i = 0; i < 7; i++) {
    screenTime[i] = parseFloat(prompt(`Enter screen time for day ${i + 1} (in hours):`));
    total += screenTime[i];
}
let average = total / 7;
console.log("Total Screen Time:", total, "hours");
console.log("Average Screen Time:", average.toFixed(2), "hours");
if (average > 6) {
    console.log("Warning: Average screen time exceeds healthy limit!");
}
else {
    console.log("Screen time is within healthy range.");
}
```

---

3. Functions & Pointers Write a program to:

● Swap two numbers using a user-defined function

● Implement swapping using pointers

● Explain why pass-by-reference is necessary here

➔ here is the code:

```
function swapNumbers(obj) {
    let temp = obj.a;
    obj.a = obj.b;
    obj.b = temp;
}
let numbers = {
    a: 10,
    b: 20
};
        console.log("Before Swapping:");
        console.log("A =", numbers.a);
        console.log("B =", numbers.b);
                swapNumbers(numbers);
        console.log("After Swapping:");
        console.log("A =", numbers.a);
        console.log("B =", numbers.b);
```

---

4. File handling Build a program that:

● Writes daily goals to a file

● Reads and displays them on program restart

● **Uses correct file modes**

➔ here is the code:

```
const fs = require("fs");

let goal = "Complete JavaScript practice and revise loops.\n";

fs.writeFileSync("goals.txt", goal, { flag: "a" });

console.log("Goal saved successfully!");

let data = fs.readFileSync("goals.txt", "utf8");

console.log("\nSaved Goals:");

console.log(data);
```

Section C: Mini Capstone Project

Mini Project: Student Productivity Tracker

Objective

Build a console-based Student Productivity Tracker that logs daily study hours and generates a weekly report.

Your application must:

1. Menu-driven program
2. Log daily study hours
3. Generate weekly report
4. Store data using files

```jsx
import React, { useState, useEffect } from "react";

function App() {
  const [studyHours, setStudyHours] = useState("");
  const [day, setDay] = useState("");
  const [records, setRecords] = useState([]);

  useEffect(() => {
    const savedData = JSON.parse(localStorage.getItem("studyRecords")) || [];
```

```
    setRecords(savedData);
  }, []);

  const addRecord = () => {
    if (day === "" || studyHours === "") {
      alert("Please enter all fields");
      return;
    }

    const newRecord = {
      day: day,
      hours: parseFloat(studyHours)
    };

    const updatedRecords = [...records, newRecord];

    setRecords(updatedRecords);

    localStorage.setItem(
      "studyRecords",
      JSON.stringify(updatedRecords)
    );

    setDay("");
    setStudyHours("");
  };

  const totalHours = records.reduce(
    (sum, record) => sum + record.hours,
    0
  );

  const averageHours =
    records.length > 0
      ? (totalHours / records.length).toFixed(2)
      : 0;

  return (
    <div style={styles.container}>
      <h1>Student Productivity Tracker</h1>

      <div style={styles.form}>
        <input
          type="text"
          placeholder="Enter Day"
          value={day}
          onChange={(e) => setDay(e.target.value)}
          style={styles.input}
```

```
      />

      <input
        type="number"
        placeholder="Enter Study Hours"
        value={studyHours}
        onChange={(e) => setStudyHours(e.target.value)}
        style={styles.input}
      />

      <button onClick={addRecord} style={styles.button}>
        Add Record
      </button>
    </div>

    <h2>Weekly Study Report</h2>

    <table border="1" cellPadding="10" style={styles.table}>
      <thead>
        <tr>
          <th>Day</th>
          <th>Study Hours</th>
        </tr>
      </thead>

      <tbody>
        {records.map((record, index) => (
          <tr key={index}>
            <td>{record.day}</td>
            <td>{record.hours}</td>
          </tr>
        ))}
      </tbody>
    </table>

    <h3>Total Study Hours: {totalHours}</h3>
    <h3>Average Study Hours: {averageHours}</h3>

    {averageHours >= 6 ? (
      <p style={{ color: "green" }}>
        Excellent Productivity!
      </p>
    ) : (
      <p style={{ color: "red" }}>
        Try to improve study consistency.
      </p>
    )}
  </div>
```

```
  );
}

const styles = {
  container: {
    textAlign: "center",
    marginTop: "30px",
    fontFamily: "Arial"
  },

  form: {
    marginBottom: "20px"
  },

  input: {
    padding: "10px",
    margin: "5px",
    width: "200px"
  },

  button: {
    padding: "10px 20px",
    cursor: "pointer"
  },

  table: {
    margin: "auto",
    marginTop: "20px"
  }
};

export default App;
```